Database Design

Project: E-Commerce & Order Management System (Olist) · **Group:** TarushTanishkTanishq
DBMS: PostgreSQL 16 · **Version:** 1.0 (mid-semester) · Source files: `db/01–14`, `docs/Design.md` §3–5

1. Entities, relationships and business rules

Entity	Meaning	Key relationships
STATE, ZIP_CODE	Brazilian states (27) and zip prefixes (19,177) with one cleaned coordinate each	A state contains many zips
CUSTOMER, CUSTOMER_ACCOUNT	A person (`customer_unique_id`) and the per-order account Olist issues (`customer_id`)	A person owns ≥ 1 accounts; an account places orders
SELLER	Marketplace seller located at a zip	Fulfils many order items
CATEGORY, PRODUCT	Product category (Portuguese + English name) and products	A category groups many products (optional)
ORDERS	One purchase	Contains items, is paid by payments, receives reviews
ORDER_ITEM *(weak)*	One product line, identified by `(order_id, order_item_id)`	Links order ↔ product ↔ seller
PAYMENT *(weak)*, PAYMENT_TYPE	Payments per order (sequential), payment method lookup	
REVIEW	Customer review, key `(review_id, order_id)` because `review_id` repeats	
ORDER_STATUS_LOG	Audit trail of status changes	Written only by trigger
ML_MODEL, ML_PREDICTION	Trained model versions and per-order scores	An order has 0..1 current prediction
APP_USER	Application login, mapped to a DB role; sellers link to their seller row	

Business rules enforced in the DBMS

1. Order status only moves forward: created → approved → invoiced → processing → shipped → delivered; `canceled` and `unavailable` are final (trigger).
2. Every status change is logged with old/new status, time and role (trigger).
3. Delivered orders, and orders with payments, cannot be deleted (trigger).
4. A review is accepted only after delivery (or for canceled/unavailable orders) (trigger).
5. Placing an order reduces product stock; stock can never be negative (trigger + CHECK).
6. An order with its items and payment is created atomically (stored procedure).
7. Prices > 0, freight ≥ 0, review score 1–5, coordinates inside Brazil, dates in a valid order (CHECK).
8. A seller user must be linked to a seller (CHECK); sellers see only their orders (RLS).

2. ER model

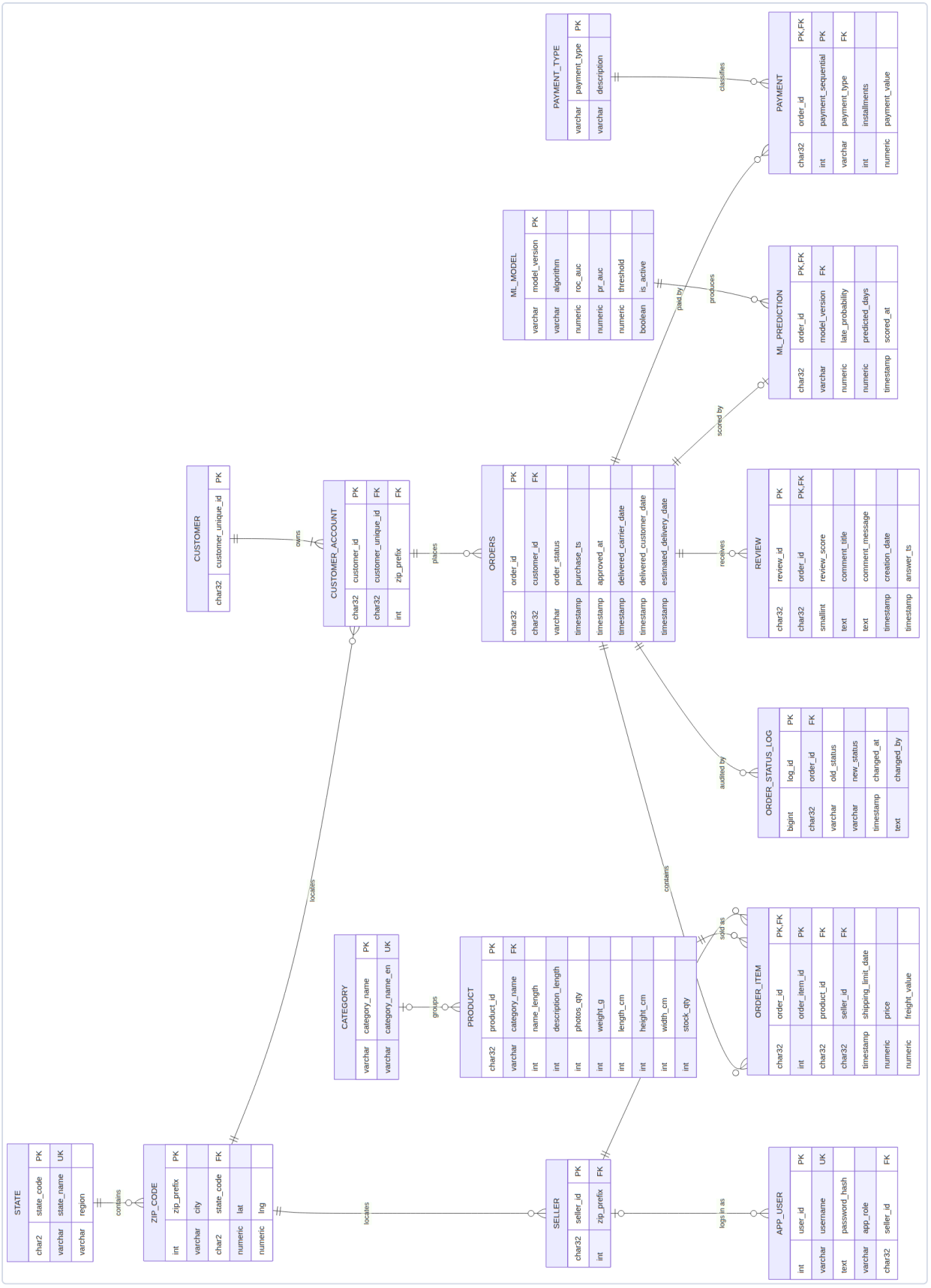


Figure 1: ER diagram, crow's-foot notation (rotated to fit the page). PK = primary key, FK = foreign key, UK = unique/candidate key. ORDER_ITEM and PAYMENT are weak entities.

EER notes. *Specialisation:* `APP_USER` is specialised by `app_role` into admin, manager, analyst, support and seller (disjoint, total); only the seller subclass has the `seller_id` attribute (enforced by `CHECK (app_role <> 'seller' OR seller_id IS NOT NULL)`). The same hierarchy is mirrored by five PostgreSQL group roles. *Weak entities:* `ORDER_ITEM` and `PAYMENT` depend on `ORDERS` (identifying relationship, `ON DELETE CASCADE`).

3. ER → relational mapping

ER construct	Mapping rule applied	Result
Strong entity	One relation, entity key = PK	`state`, `zip_code`, `customer`, `seller`, `category`, `product`, `orders`, …
Weak entity	PK = owner key + partial key; FK to owner `ON DELETE CASCADE`	`order_item(order_id, order_item_id)`, `payment(order_id, payment_sequential)`
1 : N relationship	FK on the N side	`zip_code.state_code`, `orders.customer_id`, `product.category_name`
M : N relationship (orders ↔ products ↔ sellers)	Associative relation	`order_item`
1 : 0..1 relationship	FK that is also the PK	`ml_prediction.order_id`
Multivalued / repeating data	Separate relation	payments, reviews, status log
Specialisation (app user roles)	Single relation with type attribute + CHECK	`app_user.app_role`

4. Relational schema

#	Relation	Attributes (type, NN = NOT NULL)	Primary / unique keys
1	`state`	state_code CHAR(2) NN, state_name VARCHAR(40) NN, region VARCHAR(15) NN	PK (state_code); UK (state_name)
2	`zip_code`	zip_prefix INTEGER NN, city VARCHAR(60) NN, state_code CHAR(2) NN, lat NUMERIC(9,6), lng NUMERIC(9,6)	PK (zip_prefix)
3	`customer`	customer_unique_id CHAR(32) NN	PK (customer_unique_id)
4	`customer_account`	customer_id CHAR(32) NN, customer_unique_id CHAR(32) NN, zip_prefix INTEGER NN	PK (customer_id)
5	`seller`	seller_id CHAR(32) NN, zip_prefix INTEGER NN	PK (seller_id)
6	`category`	category_name VARCHAR(60) NN, category_name_en VARCHAR(60) NN	PK (category_name); UK (category_name_en)
7	`product`	product_id CHAR(32) NN, category_name VARCHAR(60), name_length SMALLINT, description_length INTEGER, photos_qty SMALLINT, weight_g INTEGER, length_cm SMALLINT, height_cm SMALLINT, width_cm SMALLINT, stock_qty INTEGER NN	PK (product_id)
8	`orders`	order_id CHAR(32) NN, customer_id CHAR(32) NN, order_status VARCHAR(12) NN, purchase_ts TIMESTAMP NN, approved_at TIMESTAMP, delivered_carrier_date TIMESTAMP, delivered_customer_date TIMESTAMP, estimated_delivery_date TIMESTAMP NN	PK (order_id)

#	Relation	Attributes (type, NN = NOT NULL)	Primary / unique keys
9	`order_item`	order_id CHAR(32) NN, order_item_id SMALLINT NN, product_id CHAR(32) NN, seller_id CHAR(32) NN, shipping_limit_date TIMESTAMP, price NUMERIC(10,2) NN, freight_value NUMERIC(10,2) NN	PK (order_id, order_item_id)
10	`payment_type`	payment_type VARCHAR(15) NN, description VARCHAR(80) NN	PK (payment_type)
11	`payment`	order_id CHAR(32) NN, payment_sequential SMALLINT NN, payment_type VARCHAR(15) NN, installments SMALLINT NN, payment_value NUMERIC(10,2) NN	PK (order_id, payment_sequential)
12	`review`	review_id CHAR(32) NN, order_id CHAR(32) NN, review_score SMALLINT NN, comment_title TEXT, comment_message TEXT, creation_date TIMESTAMP NN, answer_ts TIMESTAMP	PK (review_id, order_id)
13	`order_status_log`	log_id BIGINT NN, order_id CHAR(32) NN, old_status VARCHAR(12), new_status VARCHAR(12) NN, changed_at TIMESTAMP NN, changed_by TEXT NN	PK (log_id)
14	`ml_model`	model_version VARCHAR(40) NN, algorithm VARCHAR(60) NN, trained_at TIMESTAMP NN, train_rows INTEGER, test_rows INTEGER, roc_auc NUMERIC(6,4), pr_auc NUMERIC(6,4), precision_at_t NUMERIC(6,4), recall_at_t NUMERIC(6,4), f1_at_t NUMERIC(6,4), threshold NUMERIC(6,4), mae_days NUMERIC(8,3), baseline_mae_days NUMERIC(8,3), is_active BOOLEAN NN, details JSONB	PK (model_version)
15	`ml_prediction`	order_id CHAR(32) NN, model_version VARCHAR(40) NN, late_probability NUMERIC(5,4) NN, predicted_days NUMERIC(6,2), scored_at TIMESTAMP NN	PK (order_id)
16	`app_user`	user_id INTEGER NN, username VARCHAR(30) NN, password_hash TEXT NN, app_role VARCHAR(20) NN, seller_id CHAR(32), is_active BOOLEAN NN	PK (user_id); UK (username)

Foreign keys

Relation	Foreign key → referenced	On delete
`zip_code`	state_code → state	RESTRICT (default)
`customer_account`	customer_unique_id → customer · zip_prefix → zip_code	CASCADE · RESTRICT
`seller`	zip_prefix → zip_code	RESTRICT
`product`	category_name → category	RESTRICT (ON UPDATE CASCADE)
`orders`	customer_id → customer_account	RESTRICT
`order_item`	order_id → orders · product_id → product · seller_id → seller	CASCADE · RESTRICT · RESTRICT
`payment`	order_id → orders · payment_type → payment_type	CASCADE · RESTRICT
`review`, `order_status_log`	order_id → orders	CASCADE
`ml_prediction`	order_id → orders · model_version → ml_model	CASCADE · RESTRICT
`app_user`	seller_id → seller	RESTRICT

5. Keys

Table	Primary key	Other candidate keys	Notes
`state`	`state_code`	`state_name`	
`category`	`category_name`	`category_name_en`	Two candidate keys: why BCNF needs checking
`customer_account`	`customer_id`	–	`customer_unique_id` is an FK (one person, many accounts)
`order_item`	`(order_id, order_item_id)`	–	Composite key of a weak entity
`payment`	`(order_id, payment_sequential)`	–	Composite key of a weak entity
`review`	`(review_id, order_id)`	–	Verified on data: `review_id` alone repeats 827 times; the composite key repeats 0 times
`app_user`	`user_id` (identity)	`username`	Surrogate key + natural candidate key

6. Functional dependencies

```
order_id                        → customer_id, order_status, purchase_ts, approved_at,
                                  delivered_carrier_date, delivered_customer_date,
estimated_delivery_date
customer_id                     → customer_unique_id, zip_prefix
zip_prefix                      → city, state_code
state_code                      → state_name, region
(order_id, order_item_id)       → product_id, seller_id, shipping_limit_date, price,
freight_value
product_id                      → category_name, name/description length, photos_qty, weight,
dimensions, stock_qty
category_name ↔ category_name_en
seller_id                       → zip_prefix
(order_id, payment_sequential)  → payment_type, installments, payment_value
(review_id, order_id)           → review_score, comment_title, comment_message, creation_date,
answer_ts
```

7. Normalization to BCNF

The starting point is the flat relation a spreadsheet export gives: one row per order line with customer, city, state, product, category and seller columns (`db/14_normalization_demo.sql` rebuilds it from the real data: **112,651 rows**).

Normal form	Problem in the flat relation	Decomposition	Result
1NF	Items and payments are repeating groups inside an order	One row per item / per payment	`order_item`, `payment`
2NF	`order_status`, `purchase_ts` depend only on `order_id`, part of key `(order_id, order_item_id)`	Move order attributes out	`orders`, `product` separated from `order_item`

Normal form	Problem in the flat relation	Decomposition	Result
3NF	Transitive dependencies `customer_id → zip_prefix → city, state` · `product_id → category_name → category_name_en` · `payment_type → description`	Move each determinant into its own relation	`zip_code`, `state`, `category`, `payment_type`
BCNF	Check that every determinant is a candidate key; `category` has two candidate keys, both determinants	Nothing further needed	All 16 relations in BCNF

Anomalies demonstrated on the real data (rolled back):

- *Redundancy:* "sao paulo, SP" is stored 17,802 times in the flat table, but once in `zip_code`.
- *Update anomaly:* renaming one category updates **11,116** flat rows, against **1** row in `category`.
- *Insert anomaly:* a new category with no sales cannot be stored in the flat table without a fake order. In the schema it is one `INSERT INTO category`.
- *Delete anomaly:* deleting a seller's only order line (e.g. the sellers in *sorocaba*, *ipira*) would lose the seller's city. In the schema it stays in `seller` / `zip_code`.
- *FD check:* `zip_prefix → city` was violated by 39 prefixes in the raw customer file (two spellings). Cleaning keeps one city per prefix, so the FD holds in `zip_code`.

8. Views

View	Type	Purpose
`v_order_details`	join view	Order + customer city, item count, order total, amount paid, rating, late flag
`v_seller_performance`	aggregate view	Orders, revenue, rating and late rate per seller
`v_monthly_sales`	aggregate view	Orders, revenue and freight per month and category (dashboard)
`v_customer_public`	security view	Customer without coordinates (support/seller access)
`v_data_quality`	exception view	Data anomalies kept visible instead of silently deleted (1,381 rows)
`v_sp_customers`	**updatable view `WITH CHECK OPTION`**	Only São Paulo customers; inserts outside SP are rejected
`v_order_features` / `mv_order_features`	view + **materialized view**	One row per order with ML features known at purchase time
`mv_seller_delivery_history`	**materialized view**	Running seller delivery history for leakage-free features
`v_high_risk_orders`	join view	Open orders joined with their late-delivery probability and features

9. Indexes (measured with `EXPLAIN ANALYZE`, `db/13_index_demo.sql`)

Index	Kind	Query it serves	Without → with index
`ix_order_item_seller`	B-tree on FK	A seller's order lines	17.9 ms (seq scan) → **2.6 ms** (bitmap index scan)

Index	Kind	Query it serves	Without → with index
`ix_orders_purchase_ts`, `ix_orders_status_ts`	B-tree, **composite** `(order_status, purchase_ts)`	Date-range reports	13.0 ms → **0.30 ms**
`ix_orders_open`	**partial** (`WHERE status NOT IN delivered/canceled/unavailable`)	Open-order dashboard	**0.06 ms**
`ix_category_en_lower`	**expression** `lower(category_name_en)`	Case-insensitive category search	–
`ix_review_fts`	**GIN** full-text (`to_tsvector('portuguese', …)`)	Review keyword search (Q16)	250.9 ms → **0.74 ms**
FK indexes (`ix_payment_order`, `ix_review_order`, `ix_orders_customer`, …)	B-tree	Joins on foreign keys	–

10. Triggers, procedures and functions

Object	Event / signature	Rule
`trg_validate_status_transition`	BEFORE UPDATE OF order_status ON orders	Forward-only status (rule 1)
`trg_order_status_log`	AFTER INSERT OR UPDATE OF order_status ON orders	Audit row (rule 2)
`trg_block_order_delete`	BEFORE DELETE ON orders	Delete guard (rule 3)
`trg_review_only_after_delivery`	BEFORE INSERT ON review	Review rule (rule 4)
`trg_reduce_stock`	AFTER INSERT ON order_item	Stock control (rule 5)
`trg_invalidate_prediction`	AFTER UPDATE ON orders	Removes a stale ML score when an order changes
`sp_place_order(customer, items jsonb, payment_type, installments, INOUT order_id)`	procedure	Order + items + payment in one transaction (rule 6)
`sp_update_status`, `sp_cancel_order`	procedures	Status change / cancel with validation
`sp_create_user`, `sp_refresh_reports`	procedures	bcrypt user creation; refresh materialized views
`fn_login`, `fn_order_total`, `fn_seller_rating`, `fn_delivery_days`, `fn_status_rank`	functions	Password check inside the DB; reusable scalar logic

11. Transactions, ACID and concurrency

Property	How it is shown	Evidence
Atomicity	`sp_place_order` with item 2 invalid → item 1 and the order row are also undone	`db/10_transactions_demo.sql`, Transactions lab, TC-12/TC-13
Consistency	Six rule violations (CHECK, FK, triggers) are rejected inside one transaction	Transactions lab "consistency", TC-03/04/06/09/10
Isolation	READ COMMITTED vs REPEATABLE READ vs SERIALIZABLE, run side by side	`db/11_concurrency_demo.sql`, Transactions lab "isolation"
Durability	COMMIT before the API answers; data survives a server restart	Standard PostgreSQL WAL
Lost update	Two concurrent sales: without `FOR UPDATE` stock 170 → 169 (lost); with `FOR UPDATE` 170 → 168	TC-27
Deadlock	Two sessions lock rows in opposite order; PostgreSQL detects and aborts one	`db/11_concurrency_demo.sql`

12. Security and role-based access control

Mechanism	Implementation
Group roles	`olist_admin`, `olist_manager`, `olist_analyst`, `olist_seller`, `olist_support` (NOLOGIN)
Least-privilege login	The app connects as `olist_app` (LOGIN, NOINHERIT, no data privileges); each request runs `SET LOCAL ROLE olist_<role>`
Object privileges	`REVOKE ALL … FROM PUBLIC`, then per-role GRANTs on tables, views, procedures
Column privileges	Analyst/support/seller may read only `zip_prefix, city, state_code` of `zip_code` (no lat/lng); support may update only `review.answer_ts`
Row-level security	Policies on `orders`, `order_item`, `review`: a seller sees only rows linked to `current_setting('app.seller_id')`
Passwords	bcrypt via `pgcrypto` (`crypt()` / `gen_salt('bf')`); the hash never leaves the database (`fn_login`)
SQL injection	All values bound as parameters; IDs must match `^[0-9a-zA-Z_]{1,32}$`